

Fasttrack — Guía completa de despliegue con Docker y ambientes QA / Producción / Deploy

levantar un sistema con bases de datos independientes usando **Docker Compose**.

Incluye la configuración de los entornos **QA (Quality Assurance)**, **Producción** y **Deploy (Staging)**.

Arquitectura general de microservicios

El proyecto se compone de **6 microservicios**:

Servicio	Descripción	Puerto
Client-service	Gestión de clientes	8081
Shipment-service	Control de envíos	8082
Fleet-service	Administración de flota	8083
Pay-service	Procesamiento de pagos	8084
Notification-service	Envío de notificaciones	8085
People-service	Gestión de personal	8086

Cada uno con su propia **base de datos MySQL** para garantizar independencia y escalabilidad.

Preparación del entorno

1. Instalar Docker y Docker Compose

Verifica si ya los tienes instalados:

```
docker --version
docker-compose --version
```

Si no están instalados, sigue las guías oficiales:

- [Instalar Docker Desktop \(Windows/Mac\)](#)
 - [Instalar Docker Engine \(Linux\)](#)
-

2. Estructura del proyecto

```
project/
├── Client-service/
├── Shipment-service/
├── Fleet-service/
├── Pay-service/
├── Notification-service/
├── People-service/
├── docker/
│   └── docker-compose.yml
```

- docker-compose.qa.yml
- docker-compose.prod.yml
- docker-compose.deploy.yml

Archivo base `docker-compose.yml`

Este archivo define todos los microservicios y sus bases de datos.
Se usa principalmente para el **entorno local o desarrollo**.

```
version: '3.9'

services:
  mysql-client:
    image: mysql:8.0
    container_name: mysql-client
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: client_db
      MYSQL_USER: client_user
      MYSQL_PASSWORD: client_pass
    ports:
      - "3307:3306"
    volumes:
      - client_data:/var/lib/mysql
    networks:
      - backend_network

  client-service:
    build: ./Client-service
    container_name: client-service
    depends_on:
      - mysql-client
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql-client:3306/client_db
      SPRING_DATASOURCE_USERNAME: client_user
      SPRING_DATASOURCE_PASSWORD: client_pass
      SERVER_PORT: 8081
    ports:
      - "8081:8081"
    networks:
      - backend_network

networks:
  backend_network:
    driver: bridge

volumes:
  client_data:
```

Levantamiento y apagado de servicios

Levantar todo el sistema

```
docker-compose up -d
```

Ver contenedores activos

```
docker ps
```

Apagar todos los servicios

```
docker-compose down
```

Apagar o reiniciar un servicio específico

```
docker stop client-service  
docker restart shipment-service
```

Gestión de ambientes (QA, Producción, Deploy)

Para cada entorno se usan distintos archivos `docker-compose`.

Esto permite cambiar credenciales, nombres de base de datos, puertos y perfiles de **Spring Boot**.

`docker-compose.qa.yml`

```
version: '3.9'

services:
  mysql-client-qa:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: qa_root
      MYSQL_DATABASE: client_db_qa
      MYSQL_USER: qa_user
      MYSQL_PASSWORD: qa_pass
    ports:
      - "4307:3306"
    networks:
      - qa_network

  client-service:
    build: ./Client-service
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql-client-qa:3306/client_db_qa
      SPRING_DATASOURCE_USERNAME: qa_user
      SPRING_DATASOURCE_PASSWORD: qa_pass
      SPRING_PROFILES_ACTIVE: qa
    ports:
      - "9081:8081"
    depends_on:
      - mysql-client-qa
    networks:
      - qa_network

networks:
  qa_network:
    driver: bridge
```

`docker-compose.prod.yml`

```
version: '3.9'
```

```
services:
  mysql-client-prod:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: prod_root
      MYSQL_DATABASE: client_db
      MYSQL_USER: prod_user
      MYSQL_PASSWORD: prod_pass
    ports:
      - "5307:3306"
    networks:
      - prod_network
    volumes:
      - client_prod_data:/var/lib/mysql

  client-service:
    image: myrepo/client-service:latest
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql-client-prod:3306/client_db
      SPRING_DATASOURCE_USERNAME: prod_user
      SPRING_DATASOURCE_PASSWORD: prod_pass
      SPRING_PROFILES_ACTIVE: prod
    ports:
      - "8081:8081"
    depends_on:
      - mysql-client-prod
    networks:
      - prod_network
    restart: always

networks:
  prod_network:
    driver: bridge

volumes:
  client_prod_data:
```

`docker-compose.deploy.yml`

```
version: '3.9'

services:
  mysql-client-deploy:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: deploy_root
      MYSQL_DATABASE: client_db_stg
      MYSQL_USER: deploy_user
      MYSQL_PASSWORD: deploy_pass
    ports:
      - "6307:3306"
    networks:
      - deploy_network

  client-service:
    image: myrepo/client-service:staging
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql-client-deploy:3306/client_db_stg
```

```
    SPRING_DATASOURCE_USERNAME: deploy_user
    SPRING_DATASOURCE_PASSWORD: deploy_pass
    SPRING_PROFILES_ACTIVE: staging
  ports:
    - "7081:8081"
  depends_on:
    - mysql-client-deploy
  networks:
    - deploy_network

networks:
  deploy_network:
    driver: bridge
```

Perfiles de Spring Boot por ambiente

Cada microservicio usa distintos archivos de propiedades según el entorno.

`application.properties`

```
spring.profiles.active=${SPRING_PROFILES_ACTIVE:dev}
```

`application-qa.properties`

```
spring.datasource.url=jdbc:mysql://mysql-client-qa:3306/client_db_qa
spring.datasource.username=qa_user
spring.datasource.password=qa_pass
logging.level.root=INFO
```

`application-prod.properties`

```
spring.datasource.url=jdbc:mysql://mysql-client-prod:3306/client_db
spring.datasource.username=prod_user
spring.datasource.password=prod_pass
logging.level.root=WARN
```

`application-staging.properties`

```
spring.datasource.url=jdbc:mysql://mysql-client-deploy:3306/client_db_stg
spring.datasource.username=deploy_user
spring.datasource.password=deploy_pass
logging.level.root=DEBUG
```

Ejecución según el entorno

Entorno	Comando
QA	<code>docker-compose -f docker/docker-compose.qa.yml up -d</code>
Producción	<code>docker-compose -f docker/docker-compose.prod.yml up -d</code>
Deploy (Staging)	<code>docker-compose -f docker/docker-compose.deploy.yml up -d</code>

-
- Usa `docker-compose logs -f` para ver los registros en tiempo real.
 - Si cambias el código del servicio, vuelve a construirlo con:

```
docker-compose build
```

- Asegúrate de usar volúmenes separados por entorno para evitar conflictos entre bases de datos.

Autor: Equipo Fasttrack

Última actualización: Octubre 2025